



UNIVERSITY OF GENEVA

Faculty of Science

Exercise #6

Introduction to Computational Finance
14X030

Michel Jean Joseph Donnet



Contents

1 First steps	3
2 TWAP algorithm	3
3 VWAP algorithm	7
4 Algorithm based on price evolution	9
5 Impact of Market Volatility	12



1 First steps

The goal of this series is to implement different execution strategies and to see the different prices we would have obtained on real data. In the following exercises, we consider a fictitious order to buy **12 million euros**.

We will again use the file `eur_usd_20120101_20120301.txt` (downloadable from Moodle, TP_03) that contains data on the EUR/USD exchange rate from January 1st to March 1st 2012 and has the following data structure:

```
timestamp  bid  ask
```

Similarly to the previous TPs, load this file and remove the outliers (date in 1970).

Then, pick several random time-intervals from the data (at least twenty). We will use them in the following exercises.

```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import numpy as np
import datetime

np.random.seed(2026)

data = np.loadtxt("eur_usd_20120101_20120301.txt")
dates = np.array([datetime.datetime.fromtimestamp(x) for x in data[:, 0]])
# Remove outliers
mask = dates > datetime.datetime(2011, 11, 11)
dates = dates[mask]
bids = data[mask, 1]
asks = data[mask, 2]
mid_prices = (bids + asks) / 2
random_indices = np.random.choice(np.arange(len(dates)), size=20,
replace=False)
```

2 TWAP algorithm

- For each of the twenty starting times you picked, simulate the execution of the aforementioned order with the **TWAP** algorithm, using 12 slices executed every 15 minutes.

```
def time_indexer(
    dates: np.ndarray, start_date: datetime.datetime, slice_interval: int =
    15
) -> np.ndarray:
    results = np.arange(len(dates), dtype=int)
    mask = (dates >= start_date) & (
        dates < start_date + datetime.timedelta(minutes=slice_interval)
```



```

    )
    results = results[mask]
    return results

def twap(
    dates: np.ndarray,
    mid_prices: np.ndarray,
    start_idx: int,
    num_slices: int = 12,
    slice_interval: int = 15,
) -> np.ndarray:
    prices = []
    for i in range(num_slices):
        idx = time_indexer(
            dates,
            dates[start_idx] + datetime.timedelta(minutes=i *
slice_interval),
            slice_interval,
        )
        if len(idx) > 0:
            prices.append(np.mean(mid_prices[idx]))
    return np.array(prices)

twap_prices = twap(dates, asks, random_indices[0]) # Example for the first
random index

dates_for_plot = [
    dates[random_indices[0]] + datetime.timedelta(minutes=i * 15) for i in
range(12)
]
idx_for_decision_price = time_indexer(
    dates, dates[random_indices[0]], slice_interval=12 * 15
)

plt.figure()
plt.plot(dates_for_plot, twap_prices, label="TWAP Execution Prices")
plt.gca().xaxis.set_major_locator(mdates.HourLocator())
plt.gca().xaxis.set_minor_locator(mdates.MinuteLocator(interval=15))
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%y-%m-%d %H:%M"))
plt.title("TWAP execution prices")
plt.xlabel("Time")
plt.ylabel("Price")
plt.gcf().autofmt_xdate()
plt.tight_layout()
plt.legend()
plt.show()

```

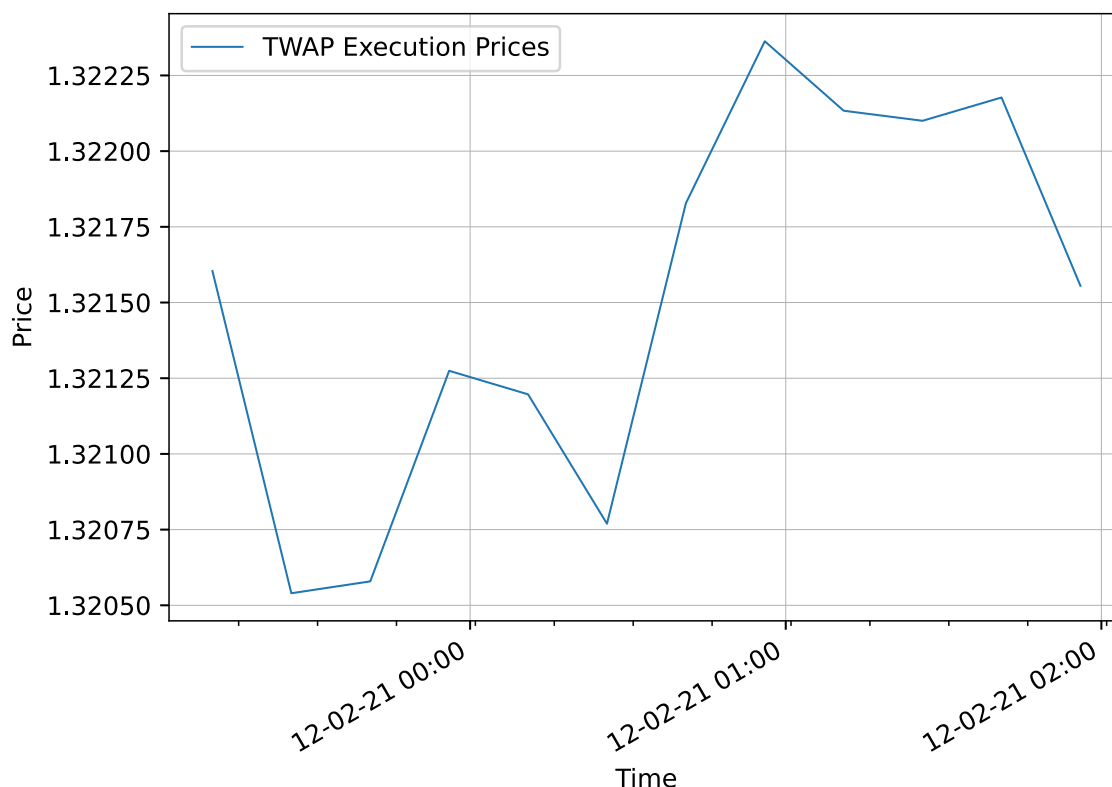


Figure 1: TWAP execution prices

- Compute the execution prices you got and compare them to the decision prices $\{\text{ask}\}(t_i)$, $\forall i \in \{1, \dots, 20\}$.

```
# Printing the difference between TWAP execution prices and decision price
for the first random index
decision_price = asks[random_indices[0]]
realized_prices = twap(dates, asks, random_indices[0])
print(f"Overall price payed with decision price: {decision_price *
12e6:.2f}")
print(f"Overall price payed with TWAP: {np.sum(realized_prices * 12e6 /
12):.2f}")
print(
    f"Difference: {(np.sum(realized_prices * 12e6 / 12) - decision_price *
12e6):.2f}"
)
```

```
Overall price payed with decision price: 15877920.00
Overall price payed with TWAP: 15858120.61
Difference: -19799.39
```

We can see on the first random index that the TWAP execution price is lower than the decision price, allowing us to save around 20,000 euros on the execution of the order which is not negligible.

If we plot the difference between the TWAP execution prices and decision prices for all the random indices as follows, we can see that the TWAP execution prices are generally



lower than the decision prices since the bars are mostly negative. This is due to the TWAP algorithm that mitigates market impact by spreading the order over time, allowing to take advantage of price fluctuations and potentially obtain better prices than the initial decision price. However, some times, the TWAP execution price can be higher than the decision price due to the market behavior during the execution period. This is shown by the few positive bars in the plot.

So the TWAP algorithm is quite effective, but can lead to worse execution prices depending on the market.

Another problem with this algorithm is that some concurrent companies can study the order flow, predict the execution of the orders and take advantage of it, which can lead to worse execution prices for the TWAP algorithm.

```
differences = np.zeros(20)
for i in range(20):
    twap_price = twap(dates, asks, random_indices[i]) * 12e6 / 12
    twap_price = np.sum(twap_price)
    decision_price = asks[random_indices[i]] * 12e6
    differences[i] = twap_price - decision_price
plt.figure()
plt.bar(range(20), differences, label="TWAP Execution Price - Decision Price")
plt.title("TWAP Execution Prices vs Decision Prices")
plt.xlabel("Random Index")
plt.xticks(range(20), [f"Index {i}" for i in range(1, 21)], rotation=45)
plt.ylabel("Price")
plt.legend()
plt.show()
```

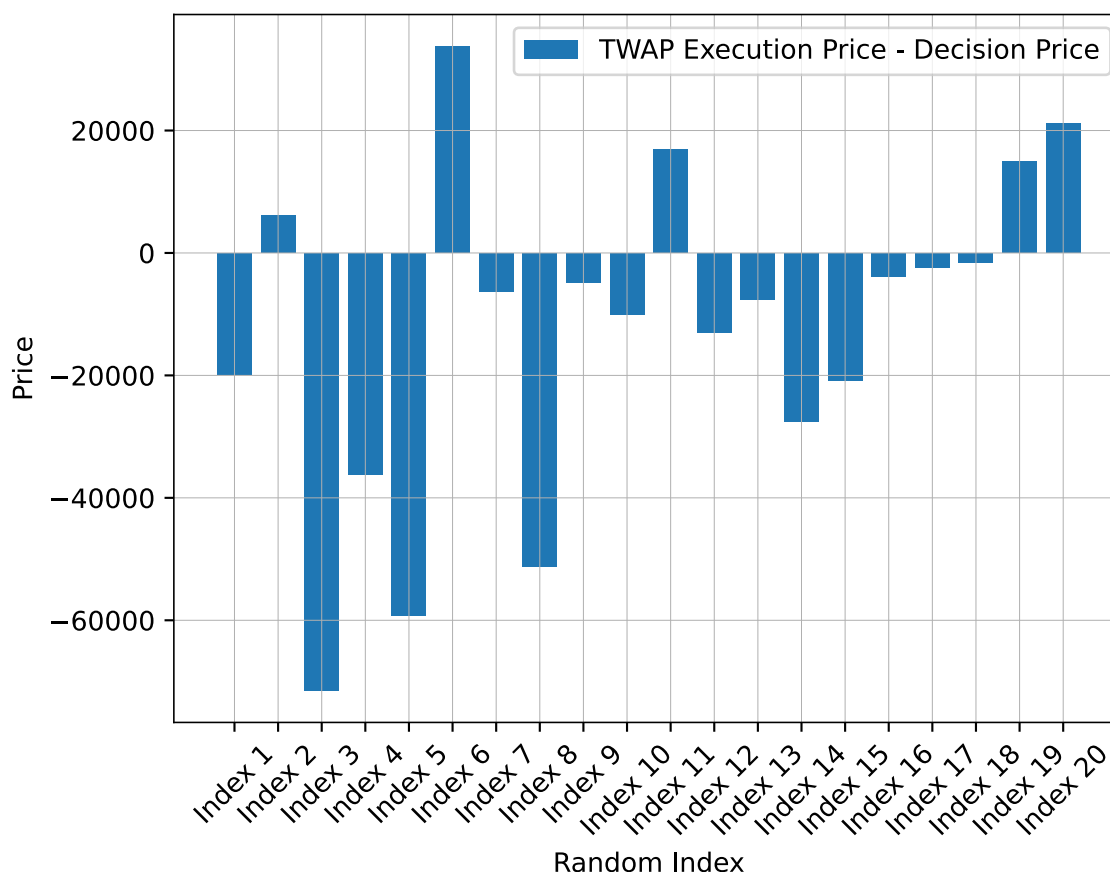


Figure 2: TWAP Execution Prices vs Decision Prices

3 VWAP algorithm

- Picking the same twenty starting times, simulate the execution of the aforementioned order with the **VWAP** algorithm, using 12 slices executed every 15 minutes. You can use the daily distribution of ticks as historical data to estimate volumes traded in each 15-minute interval.

```
def get_previous_day_volume_distribution(
    dates: np.ndarray, random_index: int, slice_interval: int = 15,
    num_slices: int = 12
) -> np.ndarray:
    volume_distribution = np.zeros(num_slices)
    start_time = dates[random_index] - datetime.timedelta(days=1)
    for i in range(num_slices):
        mask_slice = (
            dates >= start_time + datetime.timedelta(minutes=i *
            slice_interval)
            & (dates < start_time + datetime.timedelta(minutes=(i + 1) *
            slice_interval))
        )
        volume_distribution[i] = np.sum(mask_slice)
    volume_distribution /= np.sum(volume_distribution) # Normalize to get
```



```

distribution
    return volume_distribution

def vwap(
    dates: np.ndarray,
    prices: np.ndarray,
    random_index: int,
    num_slices: int = 12,
    slice_interval: int = 15,
) -> tuple[np.ndarray, np.ndarray]:
    volume_distribution = get_previous_day_volume_distribution(
        dates, random_index, slice_interval, num_slices
    )
    slice_volumes = volume_distribution
    execution_prices = twap(
        dates, prices, random_index, num_slices, slice_interval
    ) # Get TWAP prices for the slices
    return execution_prices, slice_volumes

```

- Compute the execution prices you got and compare them to the decision prices and the TWAP prices.

The VWAP execution price follows a similar pattern to the TWAP algorithm, but it takes into account the activity in the market by weighting the execution prices according to the volume distribution of previous days. In this way, the VWAP algorithm is less predictable than the TWAP algorithm for a concurrent company, which can lead to better execution prices.

If we look at the difference between VWAP and TWAP execution prices, we can see that the TWAP execution tends to be better than the VWAP execution by having more negative bars.

However, the VWAP execution can be better in case of unfavorable market conditions during execution period since when the bars are positive, the VWAP execution price is better than the TWAP execution price.

```

differences_vwap = np.zeros(20)
differences_twap = np.zeros(20)
for i in range(20):
    vwap_prices, vwap_volumes = vwap(dates, asks, random_indices[i])
    twap_prices = twap(dates, asks, random_indices[i]) * 12e6 / 12
    vwap_volumes *= 12e6 # Scale volumes to the total order size
    vwap_execution_price = np.sum(vwap_prices * vwap_volumes)
    twap_execution_price = np.sum(twap_prices)
    decision_price = asks[random_indices[i]] * 12e6
    differences_vwap[i] = vwap_execution_price - decision_price
    differences_twap[i] = twap_execution_price - decision_price

plt.figure()
plt.bar(range(20), differences_vwap, label="VWAP Difference")

```



```
plt.bar(range(20), differences_twap, label="TWAP Difference", alpha=0.7)
plt.title("Execution Prices vs Decision Prices")
plt.xlabel("Random Index")
plt.xticks(range(20), [f"Index {i}" for i in range(1, 21)], rotation=45)
plt.ylabel("Price")
plt.legend()
plt.show()
```

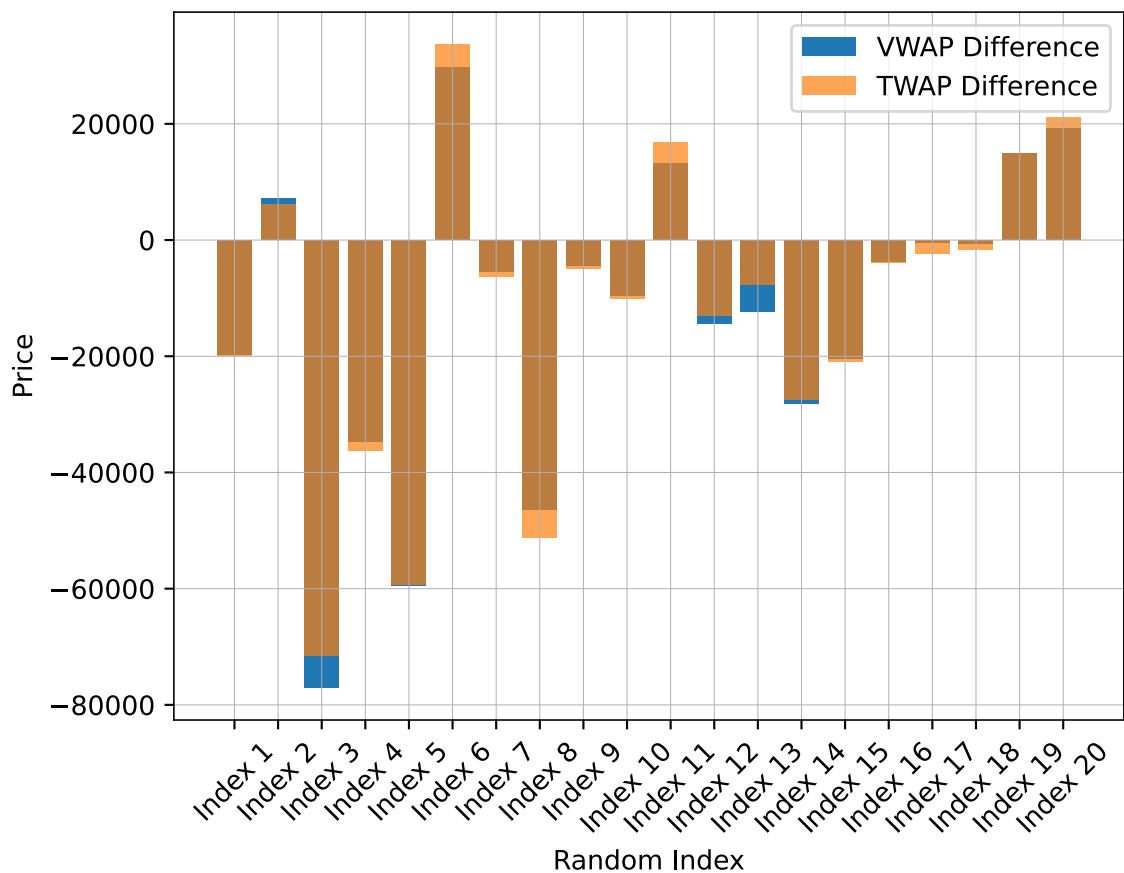


Figure 3: Execution Prices vs Decision Prices

4 Algorithm based on price evolution

Finally, implement an execution algorithm that takes into account the price evolution. For instance, you can split the order into 12 equal slices and trade a slice at each downward directional change δ .

- Picking again the same initial times, compute the execution prices you got with this method and compare them with the ones previously obtained.

```
def price_evolution_based(
    dates: np.ndarray,
    prices: np.ndarray,
```



```

    random_index: int,
    num_slices: int = 12,
    delta: float = 0.001,
) -> np.ndarray:
    execution_prices = []
    last_price = prices[random_index]
    slices_executed = 0
    for i in range(random_index, len(dates)):
        if slices_executed >= num_slices:
            break
        elif prices[i] < last_price - delta:
            execution_prices.append(prices[i])
            last_price = prices[i]
            slices_executed += 1
        elif prices[i] > last_price:
            last_price = prices[i]
    assert (
        len(execution_prices) == num_slices
    ), f"Not enough price drops to execute all slices (actual:
{len(execution_prices)}). Try reducing delta."
    return np.array(execution_prices)

differences_price_evolution = np.zeros(20)
differences_vwap = np.zeros(20)
differences_twap = np.zeros(20)
for i in range(20):
    decision_price = asks[random_indices[i]] * 12e6

    price_evolution_prices = (
        price_evolution_based(dates, asks, random_indices[i]) * 12e6 / 12
    )
    price_evolution_execution_price = np.sum(
        price_evolution_prices
    ) # Assuming equal slices
    differences_price_evolution[i] = price_evolution_execution_price -
decision_price

    vwap_prices, vwap_volumes = vwap(dates, asks, random_indices[i])
    vwap_volumes *= 12e6
    vwap_execution_price = np.sum(vwap_prices * vwap_volumes)
    differences_vwap[i] = vwap_execution_price - decision_price

    twap_prices = twap(dates, asks, random_indices[i]) * 12e6 / 12
    twap_execution_price = np.sum(twap_prices)
    differences_twap[i] = twap_execution_price - decision_price

bar_width = 0.25
bar_shift = 0.25
plt.figure()
plt.bar(
    np.arange(1, 21) - bar_shift,
    differences_price_evolution,

```



```

    label="Price Evolution Based Difference",
    width=bar_width,
)
plt.bar(
    np.arange(1, 21),
    differences_vwap,
    label="VWAP Difference",
    alpha=0.7,
    width=bar_width,
)
plt.bar(
    np.arange(1, 21) + bar_shift,
    differences_twap,
    label="TWAP Difference",
    alpha=0.7,
    width=bar_width,
)
plt.title("Execution Prices vs Decision Prices")
plt.xlabel("Random Index")
plt.xticks(range(20), [f"Index {i}" for i in range(1, 21)], rotation=45)
plt.ylabel("Price")
plt.legend()
plt.show()

```

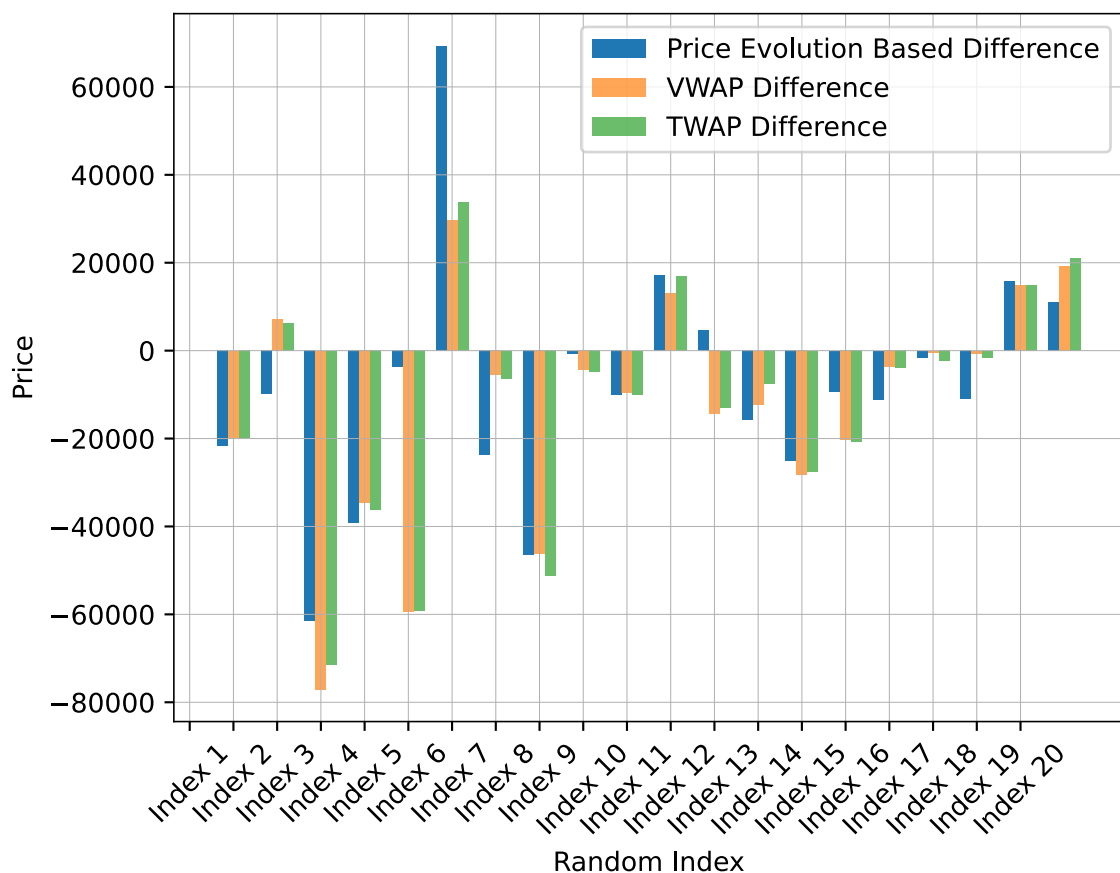


Figure 4: Execution Prices vs Decision Prices



We can observe that the price-evolution-based method can sometimes lead to better execution prices than the TWAP and VWAP algorithms, like in the case of the random index 18 of the plot. This is probably due to the exploitation of price drops that has occurred during the execution period: indeed, the price-evolution-based method allows to exploit these price drops. However, if there is not enough price drops or if there is some unfavorable market conditions, the price-evolution-based method is similar to TWAP or VWAP method, but can also lead to worse execution prices than the TWAP and VWAP algorithms, like in the case of the random index 6 of the plot.

So depending on the market conditions, the price-evolution-based method can outperform or underperform the TWAP and VWAP algorithms.

Compare to the TWAP and VWAP algorithms, the price-evolution-based method has some parameters to tune such as δ . The tuning of parameters can lead to better or worse execution prices, but allows to adapt the execution strategy to the current market conditions, which is not possible with the TWAP and VWAP algorithms that are more rigid.

- How should you choose δ to execute the full order over 3 hours?

To execute the full order over 3 hours, we need to find the right balance for δ that allows to execute all 12 slices. A smaller δ will lead to more frequent executions, while a larger δ will lead to fewer executions.

To choose δ , we can analyze the historical price data to determine the general behavior of the market and the frequency of price changes. In this way, the orders will be executed in the right timelapse.

And it is also possible to adapt δ dynamically during the execution period to have more flexibility and better adapt to the current market conditions.

5 Impact of Market Volatility

In this task, you will analyze how market volatility affects the execution prices obtained using different execution strategies (TWAP, VWAP, and the price-evolution-based method).

1. Compute Market Volatility:

- Define volatility as the standard deviation of mid-price returns over a rolling window of 30 minutes.
- Compute this measure for the selected 20 time intervals.

```
def compute_volatility(
    dates: np.ndarray, mid_prices: np.ndarray, window_size: int = 30
) -> np.ndarray:
    returns = np.diff(mid_prices) / mid_prices[:-1]
    volatility = np.zeros(len(returns))
    for i in range(window_size, len(returns)):
        volatility[i] = np.std(returns[i - window_size : i])
```



```
    return volatility

volatility = compute_volatility(dates, mid_prices)

volatility_for_intervals = np.zeros(20)
for i in range(20):
    volatility_interval = []
    for j in range(12):
        idx = time_indexer(
            dates,
            dates[random_indices[i]] + datetime.timedelta(minutes=j * 15),
            slice_interval=15,
        )
        if len(idx) > 0:
            volatility_interval.append(volatility[idx].mean())
    volatility_for_intervals[i] = np.mean(volatility_interval)
plt.figure()
plt.bar(range(20), volatility_for_intervals, label="Volatility for Intervals")
plt.title("Market Volatility for Selected Intervals")
plt.xlabel("Random Index")
plt.xticks(range(20), [f"Index {i}" for i in range(1, 21)], rotation=45)
plt.ylabel("Volatility")
plt.legend()
plt.show()
```

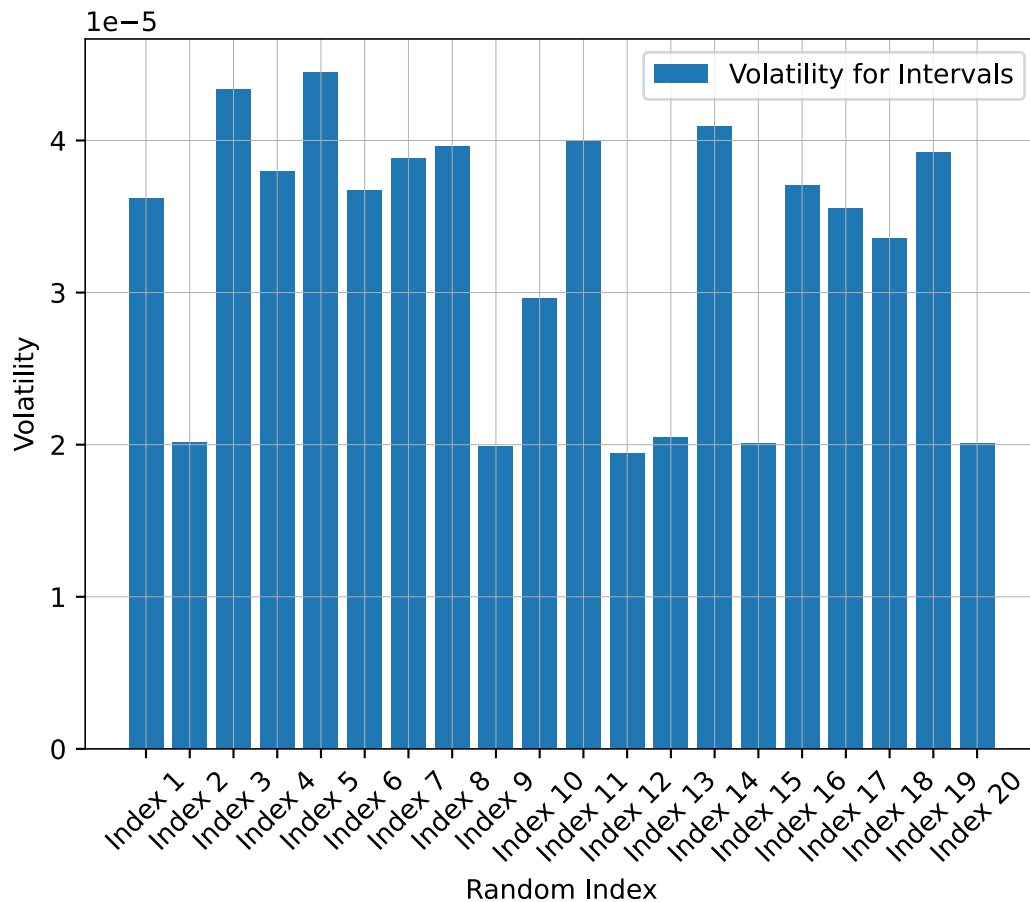


Figure 5: Market Volatility for Selected Intervals

2. Analyze Execution Performance Under Different Volatility Conditions:

- Divide the 20 intervals into two groups:
 - **Low-volatility periods:** Where the rolling standard deviation is in the bottom 50% of the dataset.
 - **High-volatility periods:** Where the rolling standard deviation is in the top 50%.
- Compute and compare the execution prices obtained using TWAP, VWAP, and price-evolution strategies across both groups. Which strategy performs best in volatile markets?

```
low_volatility_indices = np.argsort(volatility_for_intervals)[:10]
high_volatility_indices = np.argsort(volatility_for_intervals)[10:]

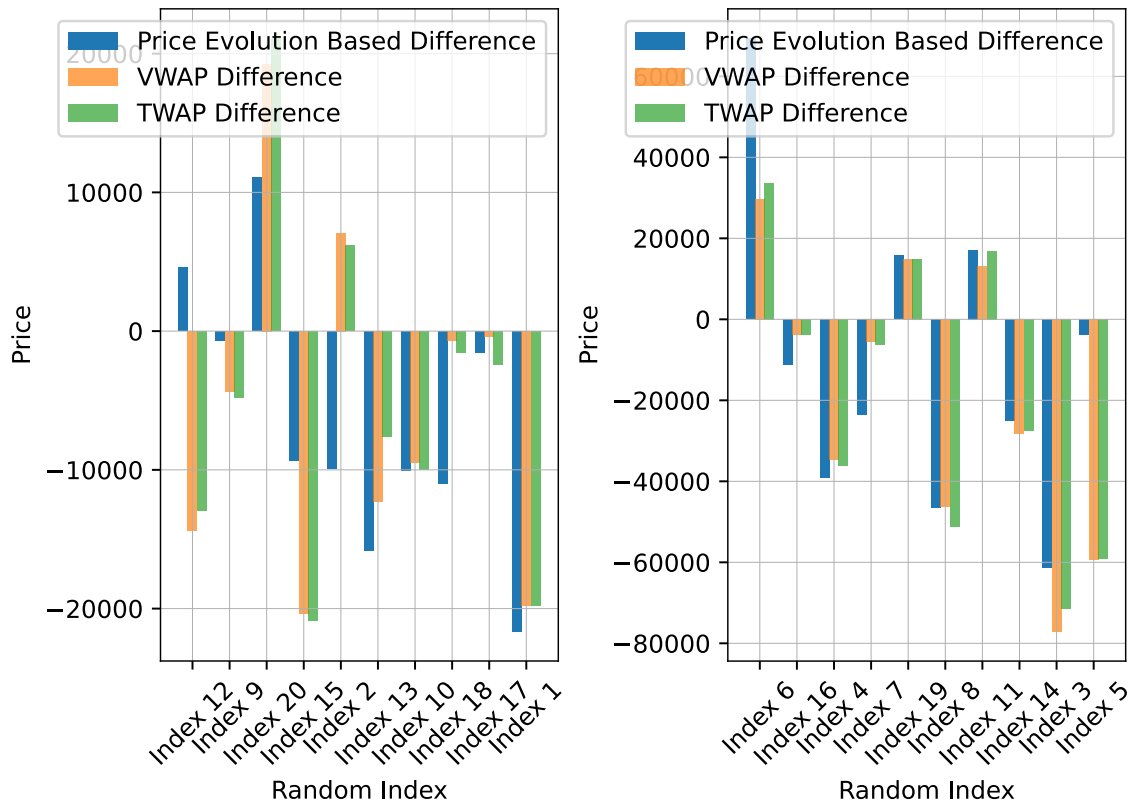
bar_width = 0.25
bar_shift = 0.25
plt.figure()
plt.suptitle("Execution Prices vs Decision Prices")
plt.subplot(1, 2, 1)
plt.bar(
    np.arange(1, 11) - bar_shift,
    differences_price_evolution[low_volatility_indices],
    label="Price Evolution Based Difference",
    width=bar_width,
```



```

)
plt.bar(
    np.arange(1, 11),
    differences_vwap[low_volatility_indices],
    label="VWAP Difference",
    alpha=0.7,
    width=bar_width,
)
plt.bar(
    np.arange(1, 11) + bar_shift,
    differences_twap[low_volatility_indices],
    label="TWAP Difference",
    alpha=0.7,
    width=bar_width,
)
plt.title("Execution Prices vs Decision Prices for Low-Volatility Periods")
plt.xlabel("Random Index")
plt.xticks(
    range(1, 11), [f"Index {i + 1}" for i in low_volatility_indices],
    rotation=45
)
plt.ylabel("Price")
plt.legend()
plt.subplot(1, 2, 2)
plt.bar(
    np.arange(1, 11) - bar_shift,
    differences_price_evolution[high_volatility_indices],
    label="Price Evolution Based Difference",
    width=bar_width,
)
plt.bar(
    np.arange(1, 11),
    differences_vwap[high_volatility_indices],
    label="VWAP Difference",
    alpha=0.7,
    width=bar_width,
)
plt.bar(
    np.arange(1, 11) + bar_shift,
    differences_twap[high_volatility_indices],
    label="TWAP Difference",
    alpha=0.7,
    width=bar_width,
)
plt.title("Execution Prices vs Decision Prices for High-Volatility Periods")
plt.xlabel("Random Index")
plt.xticks(
    range(1, 11), [f"Index {i + 1}" for i in high_volatility_indices],
    rotation=45
)
plt.ylabel("Price")
plt.legend()
plt.show()

```



(a) Execution Prices vs Decision Prices for Low-Volatility Periods (b) Execution Prices vs Decision Prices for High-Volatility Periods

Figure 6: Execution Prices vs Decision Prices

In volatile markets, the VWAP strategy perform globally better than the other strategies. This is due to the fact that the VWAP strategy takes into account the historical volume of data, which allows to better adapt to the market conditions.

However, the price-evolution-based method can outperform all the other strategies in some cases by exploiting favorable price movements during the execution period, such as in case of index 7 and index 16. But this strategy is sensitive to parameter tuning and can lead to worse execution prices if the parameters are not adapted to the market conditions, such as in case of index 5 and index 6.

- Is there a strategy that minimizes execution price deviation across all market conditions?

The best strategy to minimize execution price deviation accross all market conditions seems to be the price-evolution-based method, since it can adapt to the market conditions by exploiting favorable price movements during the execution period. For instance, this strategy outperforms the other strategies for index 2 and 13 for low-volatility periods, and for index 7 and 16 for high-volatility periods.

But to exploit this advantage, it is necessary to have a good tuning of the parameters, which can be difficult and not done properly, leading to worse execution prices than the other strategies, such as in case of index 5 and index 6 for high-volatility periods or in case of index 12 for low-volatility periods.



On top of that, the price-evolution-based method can adapt to the market conditions by making the parameters dynamic and adaptative to the current market conditions, which is not possible with the TWAP and VWAP strategies that are more rigid. But this can be more challenging to implement and to tune, and can lead to worse execution prices if not done properly.